AI ASSISTED CODING – STUDENT ASSIGNMENT

CODE REFACTORING: IMPROVING LEGACY CODE WITH AI SUGGESTIONS

Student Name: VED PRAKASH Roll No.: 2503A520Q2

Batch / Section: 67 Date: 13/10/2026

Course: AI Assisted Coding **Faculty: Saniya Sayyad Mam**

TASK 1 – Improve Readability

1. Original Code

```
a=80
b=70
c=90
x=(a+b+c)/3
if x>=50:
   print("pass")
else:
   print("fail")
```

2. AI Prompt

Refactor this code using meaningful variable names, a calculate_average() function, and clear formatting. Preserve the original behavior.

3. AI Suggestion

Use descriptive names for marks and average, move the calculation into a reusable function, and keep the pass threshold at 50.

4. Refactored Code

```
def calculate_average(mark1, mark2, mark3):
   # Calculate the average of three marks
   return (mark1 + mark2 + mark3) / 3

marks1, marks2, marks3 = 80, 70, 90
average = calculate_average(marks1, marks2, marks3)

if average >= 50:
   print("pass")
else:
   print("fail")
```

5. Test Output

pass\nAverage = 80.0 (calculated internally; the legacy program prints only pass)

6. Explanation

Meaningful names make the code easier to understand. The function separates the calculation from the decision and can be reused. The threshold and printed result are unchanged.

AI suggestion record: Accepted — descriptive names and a function improve readability. Rejected/modified — printing the average was not added, because the original code did not print it and behavior should remain equivalent.

TASK 2 – Remove Repeated Code

1. Original Code

```
name1 = "Arun"
marks1 = 85
if marks1 >= 50:
   print(name1, "Pass")
else:
   print(name1, "Fail")

name2 = "Priya"
marks2 = 72
if marks2 >= 50:
   print(name2, "Pass")
else:
   print(name2, "Fail")
```

2. AI Prompt

Identify duplicated pass/fail logic and refactor it into a reusable function. Test at least four students while preserving th e output format.

3. AI Suggestion

Create display_result(name, marks), which checks the pass threshold once and works for any student.

4. Refactored Code

```
def display_result(name, marks):
   # Print the student's result
   if marks >= 50:
      print(name, "Pass")
   else:
      print(name, "Fail")

students = [("Arun", 85), ("Priya", 72),
         ("Rahul", 40), ("Meena", 50)]

for name, marks in students:
   display_result(name, marks)
```

5. Test Output

Arun Pass\nPriya Pass\nRahul Fail\nMeena Pass

6. Explanation

The function removes repeated conditional code. If the pass rule changes, it only needs to be updated in one place. Four students test both passing and failing cases, including the boundary value 50.

AI suggestion record: Accepted — a reusable function reduces duplication. Rejected/modified — a larger class-based design was not needed for this small task; a simple function is clearer.

TASK 3 – Refactor a Long Conditional

1. Original Code

```
age = 68
temperature = 39

if age >= 60 and temperature >= 38:
   print("High Priority")
elif age >= 60 and temperature < 38:
   print("Medium Priority")
elif age < 60 and temperature >= 38:
   print("Medium Priority")
else:
   print("Normal Priority")
```

2. AI Prompt

Simplify the priority conditions into determine_priority(age, temperature). Preserve the exact original rules and test four combinations.

3. AI Suggestion

The two medium-priority conditions mean that exactly one risk factor is present. Use a function with clear, ordered conditions.

4. Refactored Code

```
def determine_priority(age, temperature):
   # Apply the original priority rules
   is_senior = age >= 60
   has_high_temperature = temperature >= 38

   if is_senior and has_high_temperature:
      return "High Priority"
   elif is_senior or has_high_temperature:
      return "Medium Priority"
   else:
      return "Normal Priority"

test_cases = [(68, 39), (68, 37), (35, 39), (35, 37)]
for age, temperature in test_cases:
   print(age, temperature, ":", determine_priority(age, temperature))
```

5. Test Output

68 39 : High Priority\n68 37 : Medium Priority\n35 39 : Medium Priority\n35 37 : Normal Priority

6. Explanation

The function names the two conditions and combines the two medium-priority cases with OR. This preserves the original behavior for all age/temperature combinations because the rules depend on whether zero, one, or both conditions are true.

AI suggestion record: Accepted — combining the medium cases reduces repetition. Rejected/modified — no new medical thresholds were introduced; the supplied thresholds remain unchanged.

TASK 4 – Refactor Inefficient List Processing

1. Original Code

```
patient_ids = [101, 205, 309, 412, 518, 623]
target = 412
found = False

for patient_id in patient_ids:
    if patient_id == target:
        found = True

if found:
    print("Patient Found")
else:
    print("Patient Not Found")
```

2. AI Prompt

Improve the readability of this patient ID search. Compare list and set lookup complexity, then implement the best choice for frequent membership checks.

3. AI Suggestion

Use target in patient_ids for a simple list membership test. If many repeated lookups are expected, convert IDs to a set for average O(1) membership lookup.

4. Refactored Code

```
patient_ids = [101, 205, 309, 412, 518, 623]
patient_id_set = set(patient_ids)  # Build once for repeated lookups

for target in (412, 999):
    if target in patient_id_set:
        print(target, "Patient Found")
    else:
        print(target, "Patient Not Found")
```

5. Test Output

412 Patient Found\n999 Patient Not Found

6. Explanation

List membership is typically O(n) because it may scan items one by one. Set membership is average O(1), after the set is built, so a set is suitable for frequent lookups. For one or very few lookups, a list may be simpler and avoids set-construction overhead.

AI suggestion record: Accepted — use a set for repeated lookups. Rejected/modified — a dictionary is unnecessary because only membership is required and no extra information is stored per ID.

TASK 5 – Complete Legacy Code Review with AI

1. Original Code

```
students = [("Arun", 78), ("Priya", 45),
           ("Rahul", 88), ("Meena", 52)]
```

```
total = 0
for s in students:
    total = total + s[1]

avg = total / len(students)

for s in students:
    if s[1] >= 50:
        print(s[0], "Pass")
    else:
        print(s[0], "Fail")

print("Average =", avg)
```

2. AI Prompt

Review this code and identify at least five improvements. Refactor it with meaningful names and reusable functions, then preserve the same student results and average.

3. AI Suggestions (at least five)

- Use descriptive variable names instead of s, total, and avg.
- Create a function to calculate the average marks.
- Create a reusable function to determine pass/fail.
- Use tuple unpacking (name, marks) instead of index access such as s[0] and s[1].
- Use sum() and a generator expression to calculate total marks clearly.
- Keep the pass threshold of 50 in one named constant.
- Test the output against the original program.

4. Refactored Code

```
PASS_MARK = 50

students = [("Arun", 78), ("Priya", 45),
            ("Rahul", 88), ("Meena", 52)]

def calculate_average(students):
    # Average the marks of all students
    total_marks = sum(marks for name, marks in students)
    return total_marks / len(students)

def get_result(marks):
    # Return pass/fail using the original threshold
    return "Pass" if marks >= PASS_MARK else "Fail"

for name, marks in students:
    print(name, get_result(marks))

average_marks = calculate_average(students)
print("Average =", average_marks)
```

5. Test Output

Arun Pass\nPriya Fail\nRahul Pass\nMeena Pass\nAverage = 65.75

6. Explanation and Review

The refactored program keeps the same pass/fail rule and average. Functions separate the average calculation and result decision. Descriptive names and tuple unpacking improve readability, while sum() reduces manual accumulation.

AI suggestion record: Accepted — reusable functions, descriptive names, tuple unpacking, sum(), and a named threshold. Rejected/modified — a class and external libraries were not added because they would make this small program more complex without a clear benefit.

Most useful change: extracting get_result() is especially useful because the pass/fail rule is defined in one place. If the threshold changes, the code is easier to maintain.

Final Verification Summary

All five refactored programs preserve the intended behavior of the supplied legacy examples. The test cases cover normal, boundary, passing, failing, found, and missing cases as relevant. Before submission, run each code block in Python to confirm the displayed output.